

Solver Take-Home Assignment

Play the real game here for context: Color Block Crush on Google Play

What is this puzzle?

Imagine a grid with colored blocks sitting on it. Each block is made of one or more cells (like Tetris pieces, but they never rotate). Colored gates sit on the edges of the board. Your job is to **slide blocks around** until every block escapes through a gate that matches its color.

This assignment asks you to write programs (solvers) that can read a level and figure out a sequence of moves that solves it.

Your task

Build **at least two different solvers** for this game. We want to see how you think about search, state representation, and performance.

Size and time limits

When we evaluate your solvers, levels will stay within these bounds:

- **Board size:** at most **12 × 12** cells.
- **Number of blocks:** at most **25** blocks per level.
- **Expected solve time:** your solver should return an answer in **under 60 seconds** per level (on a normal laptop). Faster is better, but correctness comes first.

How the game works

- **The board** is a rectangle of cells. Some cells may be **walls** — nothing can enter them.
- **Blocks** are colored shapes on the board. A block keeps its shape; it only **slides** (no rotation).
- **A move:** pick one block and slide it as far as you want along a straight horizontal or vertical line, until it hits a wall, another block, or the edge of the board.
- **Gates (exits)** are colored openings on the border. A red block can only leave through a red gate, and only when the gate is wide enough to fit the whole block along that edge.
- **Auto-exit:** right after you move, the game checks whether any block is sitting on a matching gate. If yes, that block leaves immediately. One move can cause several blocks to exit in a row.
- **You win** when every block that has a matching gate color has left the board.

Coordinates: we use (x, y) with (0, 0) at the **top-left** corner. x grows to the right; y grows downward.

Special rules (only in tests 4 and 5)

Most test levels use only the basic rules above. Two levels add extra rules:

- **Ice (ic=N):** the block is frozen. It cannot move or exit until **N other blocks** have already left. After that, it behaves like a normal block. Example: *ic=2* means “wait for 2 exits, then unlock.”
- **Directional (ar=h or ar=v):** the block can only slide in one direction — either **horizontal** (left/right) or **vertical** (up/down), not both.

How levels are given to you (ASCII format)

Each test level is a text file with **three stacked grids** (layers). Think of them as three transparent sheets placed on top of each other. Together they tell you everything about the board.

Layer 1 — COLOR

Shows the **color** of each cell. Inside the board you see one letter per block cell. On the border you see gate colors. A dot (.) means empty; a hash (#) means wall.

Layer 2 — ID

Shows **who owns each cell**. Inside the board, each block has an ID character (0, 1, 2, ... or A, B, C, ...). All cells with the same ID belong to the same block — that is the block's shape. On the border, you see **gate IDs** (a, b, c, ...) so you can tell gates apart even when two neighboring gates share the same color.

Layer 3 — MODIFIERS

On the **border**, this layer shows which way each gate faces: ^ top, v bottom, < left, > right. Inside the board, most cells are dots. If a block has ice or directional rules, you will see a short code on that block's **tag cell** (the top-left cell of the block): e.g. **i2** for ice-2, - for horizontal-only.

```
ASCII
w=6
h=6

COLOR:      (what color is each cell?)
...

ID:         (which block / gate ID is here?)
...

MODIFIERS:  (gate arrows on border; ice/directional on blocks)
...

BLOCKS:     (optional helper table)
EXITS:      (optional helper table)
```

The **BLOCKS** and **EXITS** tables at the bottom are optional helpers. You can solve a level using only the three grids if you prefer.

Quick token reference

```
Colors (layer 1):
Y Yellow  R Red      P Pink    O Orange  T Teal
B Dark blue (DB)    b Light blue (LB)
G Dark green (DG)   g Light green (LG)
. empty    # wall
```

```
Block modifiers (layer 3, on tag cell only):
iN ice — frozen until N blocks exit (e.g. i2)
- can only move horizontally
| can only move vertically
. no modifier
```

```
Gate border (layer 2 = gate id, layer 3 = direction):
ID layer border: a, b, c, ...
MODIFIERS border: ^ v < >
```

Block shapes

You do **not** need a separate shape chart. A block's shape is simply every cell in the ID layer that shares the same ID. For example, if ID **0** appears in four adjacent cells, that block is a 4-cell polyomino.

What to build

Submit **two solvers** that take different approaches. A third is welcome but not required.

- **Solver 1 — Complete:** prioritize getting the right answer. It should reliably solve levels within the size limits when a solution exists.
- **Solver 2 — Fast:** prioritize speed. It should return a good solution quickly (ideally well under 60 seconds).
- **Optional third solver:** any clearly different idea (different search, different formulation, etc.).

What to submit

- **CLI tool (solve.py)** that follows the command-line format below.
- **requirements.txt** listing all dependencies (so we can run **pip install -r requirements.txt**).
- **README** with setup steps and how to run your solvers.
- **Short write-up** (1–2 pages): state representation, move generation, and how your two solvers compare.

Command-line interface (required format)

Please use this exact interface so we can run and compare all submissions the same way. You may implement it as **python solve.py** or any language, but the flags and output must match.

How to run

```
# Solve one level file (ASCII format)
python solve.py <level.txt>

# Pick which solver to use
python solve.py <level.txt> --solver complete
python solve.py <level.txt> --solver fast

# Optional: print extra debug info to stderr (not stdout)
python solve.py <level.txt> --solver fast --verbose
```

Arguments: **<level.txt>** is required — path to a level in the ASCII format. **--solver** must accept at least **complete** and **fast** (your two solvers). Print only the solution on **stdout**; use stderr for logs.

Output format (stdout)

Print only these lines on stdout (no extra text):

```
STATUS: SOLVED | UNSOLVABLE | TIMEOUT
MOVES: <N>
<block_id> <x> <y>
... (one line per move, only when STATUS is SOLVED)
```

STATUS: SOLVED if you found a solution, UNSOLVABLE if none exists, TIMEOUT if you hit the 60s limit.

MOVES: number of moves. Each following line is one slide: block ID (from the ID layer), then the new top-left position (x, y).

Example output

```
STATUS: SOLVED
MOVES: 2
0 1 2
1 2 0
```

How we will judge your work

- **Correctness** — does the solution actually solve the level?
- **Speed** — does it finish within ~60 seconds on levels up to 12×12 and 25 blocks?
- **Move quality** — shorter solutions are a plus, but only if they are correct.

- **Code quality** — clear structure, easy to run, sensible tests.

Test cases

Five levels are included below in ASCII format. Start with Test 1 and work your way up. Tests 4 and 5 introduce ice and directional blocks.

Test 1 — Starter (no modifiers)

Small 4×5 board. Use this to check that your parser, move logic, and exit rules all work.

ASCII

w=4

h=5

COLOR:

```
. . . g g .
. . . . .
. . . g g .
. p p g g .
. p p . . .
. . . . .
. p p . . .
```

ID:

```
. . . b b .
. . . . .
. . . 1 1 .
. 0 0 1 1 .
. 0 0 . . .
. . . . .
. a a . . .
```

MODIFIERS:

```
. . . ^ ^ .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. v v . . .
```

BLOCKS:

0 LP 2x2

1 LG 2x2

EXITS:

a LP b 0 4 2

b LG t 2 0 2

Test 2 — Medium (no modifiers)

6×6 board with more blocks and gates. Good for testing search on a busier board.

ASCII

w=6

h=6

COLOR:

```
. B B . . Y Y .
. B . . . . Y .
. B B . . Y Y .
G Y . g g g B g
G Y G G G . B g
. R R R P P P .
. . . . .
. P P P R R R .
```

ID:

```
. e e . . f f .
. 7 . . . 6 .
. 7 7 . . 6 6 .
a 4 . 2 2 2 5 b
```

```

a 4 3 3 3 . 5 b
. 0 0 0 1 1 1 .
. . . . . .
. c c c d d d .

```

MODIFIERS:

```

. ^ ^ . . ^ ^ .
. . . . . .
. . . . . .
< . . . . . >
< . . . . . >
. . . . . .
. . . . . .
. v v v v v v .

```

BLOCKS:

```

0 R 3x1_H
1 P 3x1_H
2 LG 3x1_H
3 DG 3x1_H
4 Y 2x1_V
5 DB 2x1_V
6 Y RA_BR
7 DB RA_BL

```

EXITS:

```

a DG l 0 2 2
b LG r 5 2 2
c P b 0 5 3
d R b 3 5 3
e DB t 0 0 2
f Y t 4 0 2

```

Test 3 — Walls (no modifiers)

6×6 board with wall cells (#). Blocks cannot move through walls.

ASCII

```

w=6
h=6

```

COLOR:

```

. b Y Y g g P .
. R R g B Y Y .
. R g g B B Y .
. . . P b . . .
. . . b P . . .
. # Y Y g g # .
. # Y Y g g # .
. . B B R R . .

```

ID:

```

. a f f d d b .
. 4 4 6 7 8 8 .
. 4 6 6 7 7 8 .
. . . 5 2 . . .
. . . 0 1 . . .
. # 9 9 3 3 # .
. # 9 9 3 3 # .
. . e e c c . .

```

MODIFIERS:

```

. ^ ^ ^ ^ ^ ^ .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. # . . . . # .
. # . . . . # .
. . v v v v . .

```

BLOCKS:

```

0 LB 1x1
1 P 1x1
2 LB 1x1

```

```

3 LG 2x2
4 R RA_TL
5 P 1x1
6 LG RA_BR
7 DB RA_BL
8 Y RA_TR
9 Y 2x2

```

```

EXITS:
a LB t 0 0 1
b P t 5 0 1
c R b 3 5 2
d LG t 3 0 2
e DB b 1 5 2
f Y t 1 0 2

```

Test 4 — Ice + directional

Some blocks are frozen (ice) and some can only slide left/right (directional).

```

ASCII
w=6
h=6

```

```

COLOR:
. . . G G . . .
. R R B . R # .
. R R . . R R .
B R R . B B B .
. B B B . G G B
. G G . . G G .
. # G . B G G .
. . . R R . . .

```

```

ID:
. . . C C . . .
. 4 4 A . 6 # .
. 4 C . . 6 6 .
a C C . 2 9 3 .
. 1 8 0 . D D b
. 7 7 . . D 5 .
. # 7 . B 5 5 .
. . . d d . . .

```

```

MODIFIERS:
. . . ^ ^ . . .
. . . . . # .
. . i2 . . . . .
< . . . - . - .
. - . - . i2 . >
. . . . . . .
. # . . . . .
. . . v v . . .

```

```

BLOCKS:
0 DB 1x1 ar=h
1 DB 1x1 ar=h
2 DB 1x1 ar=h
3 DB 1x1 ar=h
4 R RA_TL
5 DG RA_BR
6 R RA_BL
7 DG RA_TR
8 DB 1x1
9 DB 1x1
A DB 1x1
B DB 1x1
C R RA_BR ic=2
D DG RA_TL ic=2

```

```

EXITS:
a DB l 0 2 1
b DB r 5 3 1
c DG t 2 0 2
d R b 2 5 2

```

Test 5 — Ice only

One iced block that unlocks after 1 other block exits.

ASCII

w=3

h=7

COLOR:

```
. Y Y R .  
. Y Y . .  
. Y Y . .  
. Y Y Y .  
. Y Y Y .  
. Y Y Y .  
. Y Y Y .  
. Y Y Y .  
. R Y Y .  
. . . . .
```

ID:

```
. a a b .  
. 9 9 . .  
. B B . .  
. D 6 7 .  
. C 5 8 .  
. C C 4 .  
. 1 2 3 .  
. A 0 0 .  
. . . . .
```

MODIFIERS:

```
. ^ ^ ^ .  
. i1 . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .
```

BLOCKS:

```
0 Y 2x1_H  
1 Y 1x1  
2 Y 1x1  
3 Y 1x1  
4 Y 1x1  
5 Y 1x1  
6 Y 1x1  
7 Y 1x1  
8 Y 1x1  
9 Y 2x1_H ic=1  
A R 1x1  
B Y 2x1_H  
C Y RA_BL  
D Y 1x1
```

EXITS:

```
a Y t 0 0 2  
b R t 2 0 1
```